Digital Systems

Lecture 13

Device Drivers

Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

Digital Systems

Lecture 12

Operating Systems (continued)

Mark van der Wilk

Hilary Term 2025

University of Oxford
Department of Computer Science

Following the course of past years by Mike Spivey

Mistake in Last Lecture?

What is PRIME?

```
void init(void) {
  ...
  GENPRIME = start("GenPrime", prime_task, 0, STACK);
  USEPRIME = start("UsePrime", summary_task, 1000, STACK);
}
```

Sending Messages

```
void prime_task(int arg) { int n = 2;
  message m;
  while (1) {
    if (prime(n)) {
      m.int1 = n;
      send(USEPRIME, PRIME, &m);
    }
    n++;
  }
}
```

Receiving Messages

```
void summary_task(int arg) {
  int count = 0, limit = arg; message m;
  while (1) {
    receive(PRIME, &m);
    while (m.int1 >= limit) {
      printf("There are %d primes less than %d\n",
             count, limit);
      limit += arg;
    }
    count++;
  }
}
```

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
- seeming as each program is running simultaneously,
- and doesn't require the programmer to know when to disable interrupts in order to guarantee correct code.

Message Communication

```
void init(void) {
  ...
  GENPRIME = start("GenPrime", prime_task, 0, STACK);
  USEPRIME = start("UsePrime", summary_task, 1000, STACK);
}
```

- Messages not buffered.
- If process wants to receive: It waits for the other process to send.
- If process wants to send: It waits for other process to receive.
- If message not ready, process yields, and OS runs another process.
- If message **is** ready, OS transfers when both processes are frozen.
- If two processes send to same receiver, OS determines ordering.

Message Format

Message type (2 bytes, `short`), **Sender** (2 bytes, `short`), **Data** (3 `ints`, 12 bytes)

```
typedef struct {                /* 16 bytes */
  unsigned short type;          /* Type of message */
  short sender;                 /* PID of sender */
  union {             /* An integer, a pointer, or four bytes */
    int int1; void *ptr1;
    struct { byte byte1, byte2, byte3, byte4; };
  };
  union { int int2; void *ptr2; }; /* Another int or ptr */
  union { int int3; void *ptr3; }; /* A third int or ptr */
} message;
```

Alternatives to messages

Message passing:

- no “shared variables” between processes.
- all communication by messages

Shared variables with semaphores:

- like the serial output buffer.
- more efficient, but hard to get right.

Other OSs

- Processes with communication
- Drivers for I/O devices
- Dynamically and unloading processes
- Memory management (protection/segmentation, virtual memory)
- File system
- Networking

Summary

- Why we need an OS
- How independent processes operate
- How processes can communicate by messages
- Danger of shared variables, safetey of messages

Digital Systems

Lecture 13

Device Drivers

Mark van der Wilk

Hilary Term 2025

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

List of Demands

We have a single CPU that can execute instructions.

We want to be able to

- execute multiple programs simultaneously (concurrency),
- that can respond to external events and communicate with one another,
- with each program written using its own control flow,
- seeming as each program is running simultaneously,
- **and doesn't require the programmer to know when to disable interrupts in order to guarantee correct code.**

How can we safely respond to outside events?

- Take advantage of the safety of messages, by making interrupts send messages from hardware.
- Interrupt from peripheral will send messages to *device driver*.
- Other processes communicate with hardware through device driver, by sending messages.
- Device driver is a process that serves messages from hardware/software in a loop.

Serial Output

```
void serial_putc(char ch) {
  message m;
  m.int1 = ch;
  send(SERIAL, PUTC, &m);
}
```

- Now interacts with hardware via the driver, rather than directly.
- Request message sent to SERIAL driver.
- Process yields to the OS, and waits if driver is not yet ready.

Driver Process

```
void serial_task(int arg) {
  static char txbuf[NBUF];  // Note: State is local to driver!
  int bufin, bufout, bufcount; message m; char ch;
  serial_setup();
  while (1) {                // Loop serves request
    receive(ANY, &m);
    switch (m.m_type) {
      ...
    }
  }
}
```

Setting Things Up

```
void serial_setup(void) {
  ...
  connect(UART_IRQ);
  enable_irq(UART_IRQ);
  UART_INTENSET = BIT(UART_INT_TXDRDY);
}
```

Handling PUTC Messages

```
while (1) {
  receive(ANY, &m);
  switch (m.m_type) {
    case PUTC:
      ch = m.int1;
      txbuf[bufin] = ch; ...
      break;
    ...
  }
}
```

- Buffer variables are local, so no other process can interfere.
- No need for `volatile` or to disable interrupts.

Questions

Several questions remain to be answered:

1. How do characters get out of the buffer and into the UART?
2. What happens about interrupts?
3. What happens when someone wants to send a PUTC message but the buffer is full?

Handling Interrupts

Remember: Hardware sends INTERRUPT message.

Inside loop, inside switch…

```
case INTERRUPT:
  if (UART_TXDRDY) {
    txidle = 1;
    UART_TXDRDY = 0;
  }
  clear_pending(UART_IRQ);
  enable_irq(UART_IRQ);     // Board: Why enable interrupt?
  break;
```

Responding to Events

Send character, if possible, after any message.

```
while (1) {
  receive(ANY, &m);
  switch (m.m_type) {
    ...
  }
  if (txidle && bufcount > 0) {
    UART_TXD = txbuf[bufout]; ...
    txidle = 0;
  }
}
```

Bug in the Program!

What if the buffer is full? Let's replace

```
receive(ANY, &m);
```

with

```
if (bufcount < NBUF)
    receive(ANY, &m);
else
    receive(INTERRUPT, &m);
```

When the buffer is full, we just stop accepting requests until it has emptied a bit.

Omitted Here

Lab 4 has a more elaborate serial driver

- Supports both output and input with echoing and line editing.
- All UART initialisation details are filled in.
- There's an alternative interface print_buf that overcomes the one-message-per-character bottleneck.

Bottleneck

- How many context switches per character?
- Multiple µs per context switch.
- 10k characters/s
- Can use up to 10^5 µs of CPU time per second! 10%!

This is a downside of the messaging design

- Allow `printf()` to accumulate characters in a buffer, and send them all at once.
- But… shared memory!

Standard Interrupt Handler

```c
void default_handler(void) {
  int irq = active_irq();
  int task = os_handler[irq];
  disable_irq(irq);
  interrupt(task);    // Send an interrupt message to process
}
```

- Driver can be ready to accept message
  → context switch
- Driver can not be ready to accept message
  → deliver interrupt message later, at next available opportunity

Why do we need `disable_irq(...)`?

Example

Heart & Primes example. Can we find sequence of events? When we switch contexts, when CPU sleeps/wakes?

- Look at `ex-heart.c`.
- Look at `serial.c`

If time, look at `print_buf()`.

Summary

- Take advantage of the safety of messages, by making interrupts send messages from hardware.
- Interrupt from peripheral will send messages to *device driver*.
- Device driver is a process that serves messages from hardware/software in a loop.

Summary

- Take advantage of the safety of messages, by making interrupts send messages from hardware.
- Interrupt from peripheral will send messages to *device driver*.
- Device driver is a process that serves messages from hardware/software in a loop.

? How does the OS switch tasks?

- How does it decide what to run next?
- How does it move messages?
- How does it actually do the context switch?